Lab 5 Report

과제 개요

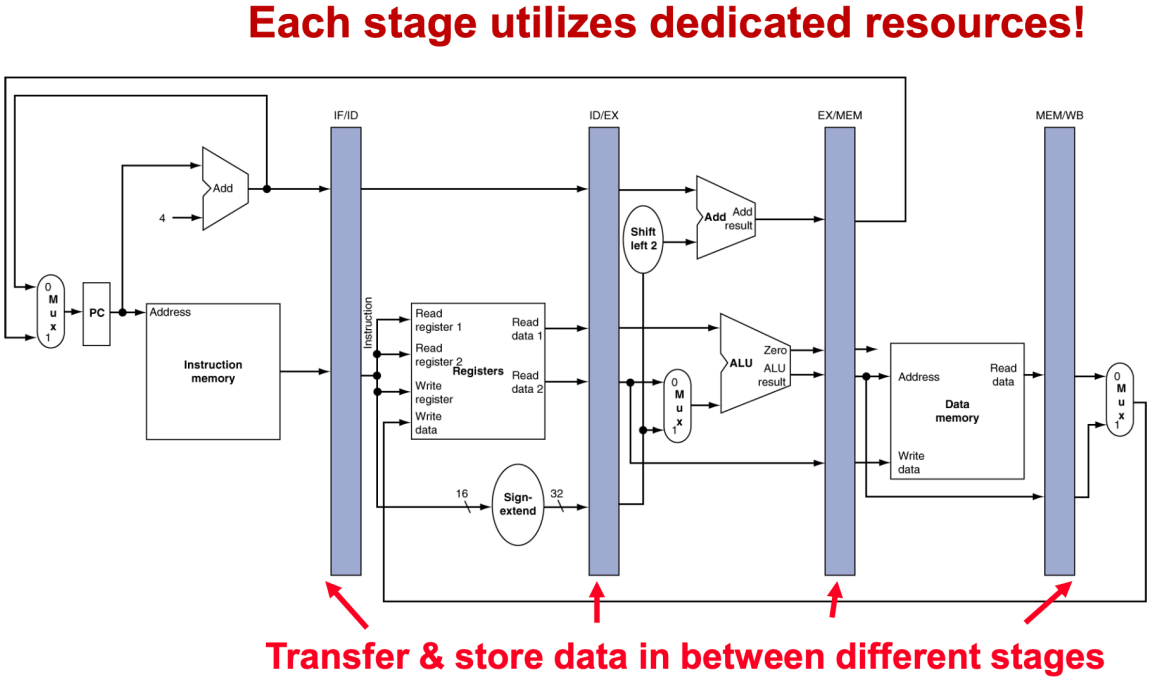
이번 과제는 HW04에서 구현하였던 Multi-cycle CPU를 파이프라인 CPU로 수정하는 과제다. 과제 명세에 따르면:

- 분기는 not taken으로 예측하고, 잘못된 예측의 경우 해당 명령어를 flush하는 방식을 사용한다.
- 데이터 해저드 탐지 시 stall을 걸고 ID단계를 재실행한다.
- 데이터 포워딩은 지원하지 않는다.

해저드 탐지와 stall 구현을 위해 추가적으로 `HAZARD.v` 파일을 생성해야 한다.

구현 과정

파이프라인 레지스터 구현



1. IF/ID 단계 파이프라인 레지스터

IF 단계에서 메모리로부터 읽어온 명령어와 계산된 PC + 4 값을 ID 단계로 전달하는 역할을 한다.

```
32      always @(posedge clk) begin
33          if (rst || IF_ID_Flush) begin
34              IF_ID_PC_plus4 <= 32'd0;
35              IF_ID_Inst <= 32'd0;
36          end else if (IF_ID_Write) begin
37              IF_ID_PC_plus4 <= PC_plus4;
38              IF_ID_Inst <= IF_Inst;
39          end
40      end
```

- `IF_ID_PC_plus4`: 분기나 JAL 명령어 주소 계산을 위해 PC + 4 값을 보존한다.
- `IF_ID_Inst`: 메모리로부터 읽어온 명령어는 ID 단계의 파싱과 컨트롤 유닛의 입력으로 들어간다.
- Stall 처리: 데이터 해저드 탐지 시 `IF_ID_Write`가 `0`이 되어 clock이 튀어도 기존 명령어를 그대로 유지하며 stall이 걸린다.
- Flush 처리: 분기 예측이 잘못되었거나 Jump 명령어가 실행될 때 `IF_ID_Flush`가 `1`이 되어 PC + 4값과 현재 읽어온 명령어를 모두 `32'd0`(nop)으로 flush시켜준다.

2. ID/EX 단계 파이프라인 레지스터

   ID 단계에서 decode된 데이터와 이후 단계에서 사용될 모든 컨트롤 신호를 모두 저장한다.

```
97      always @(posedge clk) begin
98          if (rst || ID_EX_Flush) begin
99              ID_EX_RegWrite <= 0; ID_EX_MemRead <= 0; ID_EX_MemWrite <= 0;
100             ID_EX_MemtoReg <= 0; ID_EX_RegDst <= 0; ID_EX_ALUSrcA <= 0;
101             ID_EX_ALUSrcB <= 0; ID_EX_ALUOp <= 0;
102             ID_EX_PC_plus4 <= 0; ID_EX_ReadData1 <= 0; ID_EX_ReadData2 <= 0;
103             ID_EX_SignExtImm <= 0; ID_EX_rs <= 0; ID_EX_rt <= 0; ID_EX_rd <= 0; ID_EX_shamt <= 0;
104         end else begin
105             ID_EX_RegWrite <= CTRL_RegWrite;
106             ID_EX_MemtoReg <= CTRL_MemtoReg;
107             ID_EX_MemRead  <= CTRL_MemRead;
108             ID_EX_MemWrite <= CTRL_MemWrite;
109             ID_EX_RegDst   <= CTRL_RegDst;
110             ID_EX_ALUSrcA  <= CTRL_ALUSrcA;
111             ID_EX_ALUSrcB  <= CTRL_ALUSrcB;
112             ID_EX_ALUOp    <= CTRL_ALUOp;
113             ID_EX_PC_plus4 <= IF_ID_PC_plus4;
114             ID_EX_ReadData1 <= rd_data1;
115             ID_EX_ReadData2 <= rd_data2;
116             ID_EX_SignExtImm <= SignExtImm;
117             ID_EX_rs <= rs;
118             ID_EX_rt <= rt;
119             ID_EX_rd <= rd;
120             ID_EX_shamt <= shamt;
121         end
122     end
```

- EX 단계용: `ID_EX_RegDst`, `ID_EX_ALUSrcA`, `ID_EX_ALUSrcB`, `ID_EX_ALUOp`
- MEM 단계용: `ID_EX_MemRead`, `ID_EX_MemWrite`
- WB 단계용: `ID_EX_RegWrite`, `ID_EX_MemtoReg`
- `ID_EX_PC_plus4`: PC + 4 값 저장
- `ID_EX_ReadData1`, `ID_EX_ReadData2`: 레지스터에서 읽은 값 저장
- `ID_EX_SignExtImm`: sign-extend된 immediate 값 저장
- `ID_EX_rs`, `ID_EX_rt`, `ID_EX_rd`, `ID_EX_shamt`: 명령어 필드 정보 저장

Flush 처리: 데이터 해저드로 인해 stall이 발생했을 때(`ID_EX_Flush`) 모든 컨트롤 신호와 데이터를 0으로 만들어서 명령어를 nop 처리한다.

3. EX/MEM 단계 파이프라인 레지스터

   EX 단계의 연산값과 MEM & WB 단계에서 사용될 컨트롤 신호를 저장하여 전달한다.

```
146     always @(posedge clk) begin
147         if (rst) begin
148             EX_MEM_RegWrite <= 0; EX_MEM_MemRead <= 0; EX_MEM_MemWrite <= 0;
149             EX_MEM_MemtoReg <= 0; EX_MEM_ALUResult <= 0; EX_MEM_WriteData <= 0;
150             EX_MEM_PC_plus4 <= 0; EX_MEM_DestReg <= 0;
151         end else begin
152             EX_MEM_RegWrite <= ID_EX_RegWrite;
153             EX_MEM_MemtoReg <= ID_EX_MemtoReg;
154             EX_MEM_MemRead  <= ID_EX_MemRead;
155             EX_MEM_MemWrite <= ID_EX_MemWrite;
156             EX_MEM_ALUResult <= alu_result;
157             EX_MEM_WriteData <= ID_EX_ReadData2;
158             EX_MEM_PC_plus4 <= ID_EX_PC_plus4;
159             EX_MEM_DestReg  <= EX_DestReg;
160         end
161     end
```

- MEM 단계용: `EX_MEM_MemRead`, `EX_MEM_MemWrite`
- WB 단계용: `EX_MEM_RegWrite`, `EX_MEM_MemtoReg`
- `EX_MEM_ALUResult`: ALU 연산 결과 주소 저장
- `EX_MEM_WriteData`: 메모리에 쓸 데이터 값 저장
- `EX_MEM_PC_plus4`: PC + 4 값 저장
- `EX_MEM_DestReg`: EX 단계에서 `RegDst` MUX를 통해 `rt`, `rd`, 또는 31번 (`$ra`) 중 선택된 destination 번호를 저장

4. MEM/WB 단계 파이프라인 레지스터

   레지스터 파일(RF)에 쓸 데이터와 컨트롤 신호를 저장한다.

   - `MEM_WB_MemtoReg`: 레지스터에 쓸 데이터 유형 선택용 신호 저장
   - `MEM_WB_RegWrite`: RF 파일에 쓰기 활성화 신호 저장
   - `MEM_WB_MemData`: 데이터 메모리에서 읽어온 값 저장
   - `MEM_WB_ALUResult`: ALU 연산 결과값 저장
   - `MEM_WB_PC_plus4`: JAL 복귀용 주소 저장
   - `MEM_WB_DestReg`: 쓰기를 수행할 레지스터 주소 저장

`CTRL.v`의 변경 사항

```
always @(*) begin
        RegWrite = 0; MemtoReg = 2'b00; MemRead = 0; MemWri
te = 0;
        RegDst = 2'b00; ALUSrcA = 1; ALUSrcB = 2'b00; ALUOp
```

```
= `ALU_ADDU;
        SignExtend = 0; BranchEq = 0; BranchNeq = 0; Jump =
0; JumpReg = 0;

        case(opcode)
            `OP_RTYPE: begin
                if (funct == `FUNCT_JR) begin
                    JumpReg = 1;
                end else begin
                    RegWrite = 1; RegDst = 2'b01;
                    case(funct)
                        `FUNCT_ADDU: ALUOp = `ALU_ADDU;
                        `FUNCT_SUBU: ALUOp = `ALU_SUBU;
                        ...
                `OP_LW: begin
                        ...
                        ...
                endcase
```

HW04의 multi-cycle CPU는 매 cycle마다 stage가 바뀌는 FSM이었다. 각 stage에 맞는 컨트롤 신호를 세팅해주는 방식으로 설계되었는데, 이제는 ID 단계에서 `opcode`를 보고 모든 단계의 컨트롤 신호를 한 번에 세팅하는 combinational logic으로 변경되었다. HW03의 single-cycle CPU와 컨트롤 신호 세팅 방식은 동일하다고 볼 수 있다. 여기서 세팅된 컨트롤 신호들은 위에서 구현한 파이프라인 레지스터를 통해 다음 단계로 전달된다.

자원 공유 불가

```
14      MEM imem (
15          .clk(clk),
16          .rst(rst),
17          .mem_addr(PC),
18          .MemWrite(1'b0),
19          .mem_write_data(32'b0),
20          .mem_read_data(IF_Inst)
21      );
```

명령어 메모리 `imem`

```
165     MEM mem (
166         .clk(clk),
167         .rst(rst),
168         .mem_addr(EX_MEM_ALUResult),
169         .MemWrite(EX_MEM_MemWrite),
170         .mem_write_data(EX_MEM_WriteData),
171         .mem_read_data(dmem_read_data)
172     );
```

데이터 메모리 `mem`

이제 IF 단계와 MEM 단계에서 메모리 자원을 공유할 수 없기 때문에 명령어 메모리와 데이터 메모리를 분리하였다.

해저드 제어 유닛과 스톨 구현

추가된 `HAZARD.v` 모듈과 `CPU.v`의 상호작용을 통해 해저드를 다음과 같이 처리하였다.

데이터 해저드 처리: Stall

데이터 포워딩을 지원하지 않으므로 데이터 의존성이 발생하면 앞선 명령어가 값을 레지스터에 쓸 때까지, 즉 WB 단계가 완료될 때까지 기다려야 한다.

- Stall when
  - $(rs(IR_{ID})==dest_{EX})$ && $use_rs(IR_{ID})$ && $RegWrite_{EX}$ // 3 Cycles
  - $(rs(IR_{ID})==dest_{MEM})$ && $use_rs(IR_{ID})$ && $RegWrite_{MEM}$ // 2 Cycles
  - $(rs(IR_{ID})==dest_{WB})$ && $use_rs(IR_{ID})$ && $RegWrite_{WB}$ // 1 Cycle

  - $(rt(IR_{ID})==dest_{EX})$ && $use_rt(IR_{ID})$ && $RegWrite_{EX}$ // 3 Cycles
  - $(rt(IR_{ID})==dest_{MEM})$ && $use_rt(IR_{ID})$ && $RegWrite_{MEM}$ // 2 Cycles
  - $(rt(IR_{ID})==dest_{WB})$ && $use_rt(IR_{ID})$ && $RegWrite_{WB}$ // 1 Cycle

```
24      always @(*) begin
25          PCWrite = 1; IF_ID_Write = 1; IF_ID_Flush = 0; ID_EX_Flush = 0;
26
27          if (
28              (ID_EX_RegWrite && ID_EX_rd != 0 && (ID_EX_rd == IF_ID_rs || ID_EX_rd == IF_ID_rt)) ||
29              (EX_MEM_RegWrite && EX_MEM_rd != 0 && (EX_MEM_rd == IF_ID_rs || EX_MEM_rd == IF_ID_rt)) ||
30              (MEM_WB_RegWrite && MEM_WB_rd != 0 && (MEM_WB_rd == IF_ID_rs || MEM_WB_rd == IF_ID_rt))
31             ) begin
32              PCWrite = 0;
33              IF_ID_Write = 0;
34              ID_EX_Flush = 1;
35          end
36          else if (BranchTaken || Jump || JumpReg) begin
37              IF_ID_Flush = 1;
38          end
39      end
```

현재 ID 단계에 있는 명령어의 레지스터(`IF_ID_rs`, `IF_ID_rt`)가 뒤이어 실행 중인 EX, MEM, WB 단계의 명령어의 목적지 레지스터(`ID_EX_rd`, `EX_MEM_rd`, `MEM_WB_rd`) (위의 경우 dest)와 일치하며, 해당 명령어가 레지스터에 쓰기(`RegWrite`)를 수행하는지 검사한다.

검사 결과 해저드가 탐지되면 stall을 건다.

- `PCWrite = 0`: PC 값을 업데이트하지 않아 IF 단계가 다음 명령어를 fetch하지 못하도록 막는다.
- `IF_ID_Write = 0`: 현재 ID 단계의 명령어를 그대로 유지한다.
- `ID_EX_Flush = 1`: 뒷 단계로 넘어가는 파이프라인 레지스터를 0으로 비워, EX 단계에는 nop이 들어가게 한다.

컨트롤 해저드 처리: Early Branch Resolution & Flush

```
80      wire IsEqual = (rd_data1 == rd_data2);
81      wire BranchTaken = (CTRL_BranchEq & IsEqual) | (CTRL_BranchNeq & ~IsEqual);
82
83      wire [31:0] BranchTarget = IF_ID_PC_plus4 + (SignExtImm << 2);
```

EX 단계의 ALU까지 기다리지 않고, `CPU.v`의 ID 단계에서 레지스터 파일의 출력값을 즉시 비교하여 분기 여부(`BranchTaken`)와 target address(`BranchTarget`)를 미리 계산한다.

```
86      assign next_PC = CTRL_JumpReg ? rd_data1 :
87                       CTRL_Jump ? JumpTarget :
88                       BranchTaken ? BranchTarget : PC_plus4;
```

`CPU.v`에서 `next_PC` 값을 분기 결과에 맞게 설정

```
36          else if (BranchTaken || Jump || JumpReg) begin
37              IF_ID_Flush = 1;
38          end
```

`HAZARD.v`에서 flush 구현

과제 명세에 따라 분기는 not taken으로 예측하고 일단 PC + 4 값(`PC_plus4`)을 사용하고, 이후 잘못된 예측임이 판명되면 계산해둔 target address로 세팅해주고, `IF_ID_Flush`를 `1`로 설정하여 해당 파이프라인 레지스터를 flush시킨다.

테스트 결과

```
yjkim0306 ~/Documents/3-1/컴퓨터구조론/lab/HW05 Skeleton Code main bash run_all.sh
Compiling Verilog files...
========================================
          Running Testcase 1
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 11465000 (1ps)
========================================
          Running Testcase 2
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 4135000 (1ps)
========================================
          Running Testcase 3
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 2545000 (1ps)
========================================
          Running Testcase 4
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 17785000 (1ps)
========================================
          Running Testcase 5
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5555000 (1ps)
========================================
          Running Testcase 6
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5005000 (1ps)
========================================
          Running Testcase 7
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 12795000 (1ps)
========================================
          All testcases finished!
========================================
```

7개의 테스트케이스 모두 통과하였다.